



FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

COURSE 5 - ANGULAR



BUILD AN APP WITH ANGULAR AND ANGULAR CLI

- Steps for building an App using Angular CLI on Linux:
- **Step 1:** Install Node and Node Package Manager (NPM)
 - **Install Node.js:** Download Node.js from the official website.
 - **Verify Versions:** Open your terminal and verify Node.js and NPM versions.
- **Step 2:** Install the Angular CLI
 - Run the code below to install Angular CLI:

```
npm install -g @angular/cli
```
- **Step 3:** Setting up your environment
 - In order to check whether the Angular CLI is properly installed or not, we can check for its version by typing:

```
ng -v or ng --version
```

BUILD AN APP WITH ANGULAR AND ANGULAR CLI

- **Step 4: Creating a new Angular project**

- Open the terminal and navigate to the directory where you want to create your project.
- Run the below command where "my-app" is the name of your project.
- This will create a new Angular project in a new directory called "my-app".

```
ng new Project-Name
```

- **Step 5: Navigate to project directory**

- Next, navigate into the project directory by running the following command:

```
cd Project-Name
```

- **Step 6: Run the application**

- Open the application in the browser using the following command:

```
ng serve --open
```

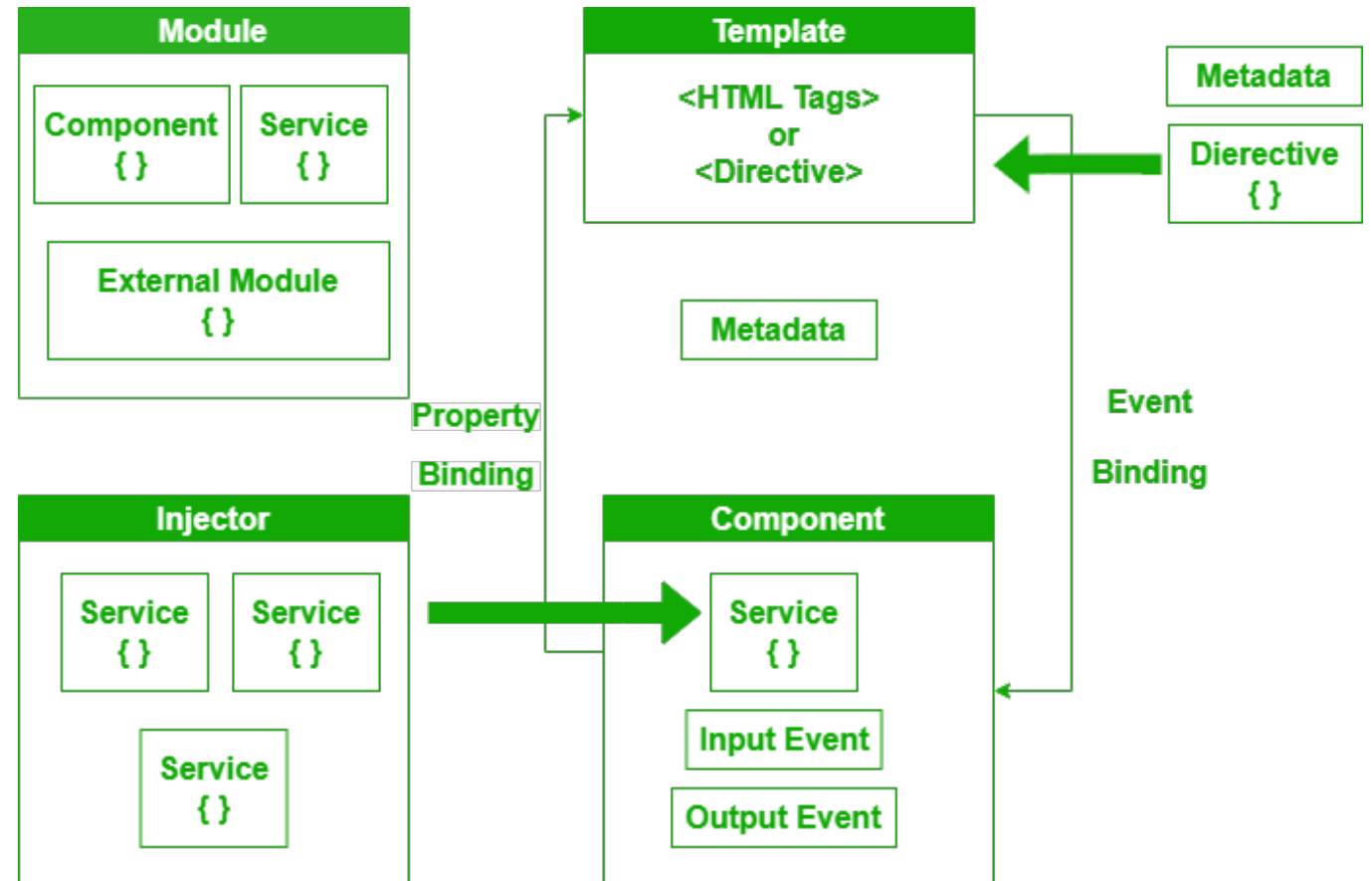
ARCHITECTURE OVERVIEW OF ANGULAR

- Angular is a client-side front-end framework developed by a team of developers at Google based on Typescript.
- Angular follows the MVC (Model-View-Controller) and MVVM (Model-View-ViewModel) design patterns.
- These types of design patterns usually maintain a clear distinction between data (Model), user interface (View), and logic (Controller/ViewModel), which results in more scalable and testable applications.
- It also provides code reusability and ensures a smoother development process for large & complex projects.

ARCHITECTURE OVERVIEW OF ANGULAR

- An Angular Application contains the following building blocks:

- Modules
- Components
- Templates
- Directives
- Services
- Dependency Injection
- Router
- State Management
- HTTP Client



ARCHITECTURE OVERVIEW OF ANGULAR

- Modules
 - A Module is a unit that consists of a separate block to perform specific functionality and enables us to break down the application into smaller chunks.
 - In a module, we can export & import the components and services from other modules.
 - Modules are created using the *@NgModule* decorator.
 - **Types of Modules:**
 1. **Core Module/Root Module**
 - Every Angular application must have at least one root module, which is called the **AppModule**, defined in the *app.module.ts* file.
 - The root module is the top-level module in an Angular application.
 - It imports all of the other modules in the application.
 2. **Feature Module**
 - Feature modules are used to group related components, directives, pipes, and services together.
 3. **Shared Module**
 - The most commonly used functionality will be present in the shared module which can be imported by the feature module whenever needed.

ARCHITECTURE OVERVIEW OF ANGULAR

```
import { NgModule } from '@angular/core';
import { BrowserModule }
  from '@angular/platform-browser';
import { AppRoutingModule }
  from './app-routing.module';
import { AppComponent } from './app.component';
@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

ARCHITECTURE OVERVIEW OF ANGULAR

- Components

- A Component is the building block of the angular application.
- A component consists of a template (HTML view of UI), styles (CSS appearance/design) and a typescript class which contains business logic.
- To indicate a class as component, `@Component` decorator is used.
- The `@Component` decorator provides metadata to the component.
- The component metadata properties consist of selectors, directives, providers, styleUrls and templateUrls.

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'AngularApp';
}
```

ARCHITECTURE OVERVIEW OF ANGULAR

- Templates

- The user interface or the view of the end users is defined using the template.
- Templates are created using HTML which binds the component properties and methods thus helping us to render data dynamically.
- Template syntax includes directives, interpolation, built-in directives, template expression operators, property binding, and event binding for creating dynamic and interactive views.

<p>Angular Architecture consists of :</p>

Modules

Components

Templates

Directives

Services

Dependency Injection(DI)

Router

HTTP Client

State Management

ARCHITECTURE OVERVIEW OF ANGULAR

- Directives

- Directives are instructions in the DOM (Document Object Model).
- Directives are used in templates to customize the behaviour of the elements.
- Angular provides built-in directives like `*ngIf` and `*ngFor`, as well as custom directives created by developers.
- **Types of directives:**
 1. **Component Directives** - These directives are associated with the template (view) of a component.
 2. **Structural Directives** - These directives are used to change the structure of the DOM using `*ngFor`, `*ngSwitch` and `*ngIf`.
 3. **Attribute Directives** - These directives are used to change the behaviour of the DOM using `ngStyle`, `ngModel` and `ngClass`.
 4. **Custom Directives** - We can create custom directives using `@Directive` decorator and define the desired behaviour in the class.

ARCHITECTURE OVERVIEW OF ANGULAR

- Services

- Services are used when specific data or logic needs to be used across different components.
- Services are typically used to centralize data access, HTTP requests, state management, and other common tasks.
- Services are singleton and are registered with Angular's dependency injection system.
- Components can inject services to access their functionality and data.
- Command to create a counter service:
`ng generate service counter`

ARCHITECTURE OVERVIEW OF ANGULAR

```
import { Injectable } from '@angular/core';
@Injectable({
  providedIn: 'root',
})
export class CounterService {
  private count = 0;
  increment(): void {
    this.count++;
  }
  getCount(): number {
    return this.count;
  }
}
```

ARCHITECTURE OVERVIEW OF ANGULAR

- Dependency Injection
 - Dependency injection simplifies dependency management, facilitates code reusability and simplifies testing.
 - DI is a design pattern which increases the flexibility and modularity of the applications by producing and distributing specific parts of the application to other parts of the application that need them.
 - We can inject services, configuration values, and other objects into components and services.
 - Components and services can declare their dependencies and have them injected automatically using *@Injectable* decorator.

ARCHITECTURE OVERVIEW OF ANGULAR

- Router
 - The Angular Router manages navigation within the application for changing from one view to another view.
 - Routes are defined in the *app-routing.module.ts* file and map to specific components.
 - The router also supports route parameters, route guards, and child routes for creating complex navigation structures.

ARCHITECTURE OVERVIEW OF ANGULAR

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule }
  from '@angular/router';
import { EmployeeListComponent }
  from './employee-list/employee-
list.component';
import { CreateEmployeeComponent }
  from './create-employee/create-
employee.component';
import { UpdateEmployeeComponent }
  from './update-employee/update-
employee.component';
import { EmployeeDetailsComponent }
  from './employee-details/employee-
details.component';
```

```
const routes: Routes = [
  { path: 'employees',
    component: EmployeeListComponent },
  { path: 'create-employee',
    component: CreateEmployeeComponent },
  { path: '', redirectTo: 'employees',
    pathMatch: 'full' },
  { path: 'update-employee/:id',
    component: UpdateEmployeeComponent },
  { path: 'employee-details/:id',
    component: EmployeeDetailsComponent }
];
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

ARCHITECTURE OVERVIEW OF ANGULAR

- State Management
 - State management in angular is achieved using RxJS (Reactive Extensions for JavaScript).
 - RxJS is used for handling asynchronous operations, such as handling HTTP requests, user interactions, and event-driven programming.
 - Streams of data and events are managed by Observables which is provided by RxJS.

ARCHITECTURE OVERVIEW OF ANGULAR

■ HTTP Client

- HTTP client module in angular is used for making HTTP requests to interact with backend services (API calls) to fetch or send data.

```
import { Component, OnInit }  
  from '@angular/core';  
import { HttpClient }  
  from '@angular/common/http';  
  
@Component({  
  selector: 'app-post-list',  
  templateUrl: './post-  
list.component.html',  
  styleUrls: ['./post-list.component.css'],  
})
```

```
export class PostListComponent implements  
OnInit {  
  posts: any[] = [];  
  
  constructor(private http: HttpClient) { }  
  
  ngOnInit(): void {  
    this.http  
      .get < any[] > (  
  
'https://jsonplaceholder.typicode.com/posts')  
      .subscribe((data) => {  
        this.posts = data;  
      });  
  }  
}
```

ANGULAR COMPONENTS

- **Angular components** are the building blocks of Angular applications.
- Each component represents a **reusable piece of UI functionality** and can be composed together to create complex applications.
- The structure of an Angular component consists of three main parts:
 1. **Template:** The template defines the HTML markup of the component's view. It contains placeholders and Angular directives that are replaced with dynamic data and logic during runtime.
 2. **Styles:** The styles define the component's visual appearance, including CSS rules and stylesheets. Styles can be defined using inline styles, external CSS files, or CSS preprocessors like Sass or Less.
 3. **TypeScript Code:** The TypeScript code defines the component's behavior and logic. It includes properties, methods, and lifecycle hooks that control how the component interacts with the DOM, handles user input, and responds to changes in its state or props.

ANGULAR COMPONENTS

- Angular components have a lifecycle consisting of various lifecycle hooks that are executed at different stages of the component's lifecycle.
- These lifecycle hooks allow to hook into specific moments in the component's lifecycle and perform actions such as initialization, cleanup, or handling changes.
- Some of the most commonly used lifecycle hooks include:
 - **ngOnInit:** Called after the component's inputs are initialized and the component's view has been initialized.
 - **ngOnChanges:** Called whenever the component's inputs change.
 - **ngOnDestroy:** Called when the component is being destroyed and cleaned up.

ANGULAR COMPONENTS

- Data binding in Angular allows for communication between the component's TypeScript code and its template.
- There are three types of data binding in Angular:
 1. **One-Way Binding:** Data flows from the component's TypeScript code to its template. Changes in the component's properties are reflected in the template, but changes in the template do not affect the component's properties.
 2. **Two-Way Binding:** Data flows both ways between the component's TypeScript code and its template. Changes in the component's properties are reflected in the template, and changes in the template are propagated back to the component's properties.
 3. **Event Binding:** Allows the template to listen for events triggered by the user or the browser and execute corresponding methods in the component's TypeScript code.

ANGULAR COMPONENTS

- Input properties allow data to be passed into a component from its parent component, while output properties allow a component to emit events to its parent component.
- Input properties are defined using the `@Input` decorator, while output properties are defined using the `@Output` decorator along with `EventEmitter`.

ANGULAR COMPONENTS

- Component communication in Angular involves passing data between components and coordinating their behavior.
- There are several methods for component communication:
 - **Parent-Child Communication:** Data can be passed from a parent component to a child component using input properties, and events can be emitted from the child component to the parent component using output properties.
 - **Sibling Communication:** Sibling components can communicate indirectly through their common parent component by passing data through input and output properties or using a shared service to store and exchange data.
 - **Communication Between Unrelated Components:** Components that are not directly related can communicate through a shared service. The shared service acts as a mediator, allowing components to exchange data and coordinate their behavior without having a direct relationship.

ANGULAR COMPONENTS

Features of components

- **Reusability:** Components promote code reusability by encapsulating functionality and UI elements.
- **Encapsulation:** Components have their own scope and encapsulate their template, styles, and behavior.
- **Maintainability:** Components make it easier to maintain and organize code by breaking it into smaller, manageable pieces.

ANGULAR COMPONENTS

- Let's create an App!
- **Step 1:** Create a new Angular app
ng new my-angular-app
- **Step 2:** Move to the Project Directory
cd my-angular-app
- **Step 3:** Generate a new component
ng generate component my-component
- **Step 4:** Edit the component files
 - Edit the generated component files (`my-component.component.html`, `my-component.component.css`, `my-component.component.ts`) to define the template, styles, and behavior of the component.

ANGULAR COMPONENTS

```
// app.component.html
```

```
<app-my-component></app-my-component>
```

```
// my-component.component.html
```

```
    <div>
      <h1>Welcome to My Component!</h1>
      <p>{{ message }}</p>
      <button (click)="changeMessage()">Change
Message</button>
    </div >
```

```
// component.component.ts
```

```
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-my-component',
  standalone: true,
  imports: [],
  templateUrl: './my-component.component.html',
  styleUrls: ['./my-component.component.css']
})
export class MyComponentComponent {
  message: string = 'Initial message';

  changeMessage() {
    this.message = 'New message';
  }
}
```

ANGULAR COMPONENTS

- Step 5: Run your Application
ng serve
- **Output:** When you run your Angular app (`ng serve`), you'll see the message "Welcome to My Component!" displayed on the screen. Clicking the "Change Message" button will update the message to "New message" dynamically.

DATA BINDING IN ANGULAR

- In Angular, binding syntax lets you determine the channel of data transmission between the component class and the template.
- Among various types of bindings supported by Angular are interpolation, property binding, event binding, and two-way-data-binding.

DATA BINDING IN ANGULAR

- **Interpolation**

- Interpolation is a communication method that links view and component through double curly braces `{{ }}`. It is used to show expression values inside the template.

```
<p>{{ title }}</p>
```

- The title component property is shown within this paragraph tag.

DATA BINDING IN ANGULAR

- **Property binding**
- Using property binding, you can tie a component field to a DOM element's property.
`<img [src]="imageUrl">`
- In this case, the `imageUrl` property of the component is connected with respect to the `img` element's `src` attribute.

DATA BINDING IN ANGULAR

- **Event Binding**

- Event Binding enables you to pay attention to events like clicks and carry out some action when they happen.

```
<button (click)="onClick()">Click me</button>
```

- When the button is clicked, the 'onClick' method is executed.

DATA BINDING IN ANGULAR

- **Two-Way Data Binding**

- A two-way binding is basically a combination of event binding and property binding which can be used to synchronize data between template and component.

```
<input [(ngModel)]="name">
```

- The property name of the component binds with a specific input field. Any changes in this input field will make changes to the name and vice versa.

DATA BINDING IN ANGULAR

- To implement binding syntax in an Angular application, follow these steps:
- **Step 1:** Create a new Angular Project
`ng new binding-syntax-example`
- **Step 2:** Install Required Modules. Ensure you have '@angular/forms' for two-way data binding
`npm install @angular/forms`
- **Step 3:** Generate a new component where you will demonstrate the different binding syntaxes
`ng generate component binding-demo`

DATA BINDING IN ANGULAR

- Next is an example of what data binding looks like in a single component.
- In the **binding-demo.component.html** file we implement interpolation, property binding and event binding:

```
// binding-demo.component.html
<h1>{{ title }}</h1> <!-- Interpolation -->
<img [src]="imageUrl" alt="Sample Image"> <!-- Property Binding -->
<br>
<button (click)="onClick()">Click me</button> <!-- Event Binding -->
<br>
<input [(ngModel)]="name">
<p>Your name: {{ name }}</p>
```

DATA BINDING IN ANGULAR

- The image URL and onClick() are defined in the **binding-demo.component.ts** file:

```
// binding-demo.component.ts
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms'; // Import FormsModule
@Component({
  selector: 'app-binding-demo',
  standalone: true,
  imports: [FormsModule], // Import FormsModule here
  templateUrl: './binding-demo.component.html',
  styleUrls: ['./binding-demo.component.css']
})
export class BindingDemoComponent {
  title = 'Binding Syntax in Angular';
  imageUrl =
'https://upload.wikimedia.org/wikipedia/commons/thumb/c/cf/Angular_full_color_logo.svg/1200px-Angular_full_color_logo.svg.png';
  name = '';
  onClick() {
    alert('Button clicked!');
  }
}
```

DATA BINDING IN ANGULAR

- The **app.component.html** file hosts the template for the custom component:

```
// app.component.html
```

```
<app-binding-demo></app-binding-demo>
```

DATA BINDING IN ANGULAR

- And finally, the app.component.ts file defines a standalone component that imports all the necessary modules and components used by the App component.

```
// app.component.ts
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { BindingDemoComponent } from '../binding-demo/binding-demo.component';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, BindingDemoComponent, FormsModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'binding-syntax-example';
}
```

BUILT-IN DIRECTIVES IN ANGULAR

- **Directives** are markers in the Document Object Model (DOM).
- Directives can be used with any controller or HTML tag which will tell the compiler what exact operation or behavior is expected.
- **There are 3 types of directives and each type has their own built-in directives.**

BUILT-IN DIRECTIVES IN ANGULAR

Component Directives

- Components are directives with templates.
- Components are used to create reusable and modular UI elements.
- They are declared using the `@Component` decorator and typically have a corresponding HTML template.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
})
export class AppComponent {}
```

BUILT-IN DIRECTIVES IN ANGULAR

Attribute Directives

- Attribute directives are used to change the appearance or behavior of a DOM element by applying custom attributes.
- These directives are applied to elements as attributes and are denoted by square brackets.
- Attribute directives are often used for tasks such as dynamic styling, input validation, or DOM manipulation.

Built-in Attribute Directives

- **ngClass:** The NgClass directive allows us to conditionally apply CSS classes to HTML elements.

```
<div [ngClass]="{'class-name': condition}">  
  <!-- Content here -->  
</div>
```

BUILT-IN DIRECTIVES IN ANGULAR

- ❑ **ngStyle:** The NgStyle directive enables you to conditionally apply inline styles to HTML elements.

```
<div [ngStyle]="{'property': 'value'}">  
  <!-- Content here -->  
</div>
```

- ❑ **ngModel:** The NgModel directive provides two-way data binding for form elements, syncing data between the model and the view.

```
<input [(ngModel)]="property">
```

BUILT-IN DIRECTIVES IN ANGULAR

```
// app.component.html
```

```
<div class="container">
```

```
  <div class="content">
```

```
    <div [ngClass]="{'highlight': isHighlighted,  
'italic': isItalic}">
```

```
      This div's classes are dynamically applied  
based on conditions.
```

```
    </div>
```

```
    <div [ngStyle]="{'color': textColor, 'font-size':  
fontSize + 'px'}">
```

```
      This div's styles are dynamically applied  
based on properties.
```

```
    </div>
```

```
    <input type="text" [(ngModel)]="username"  
class="input-field">
```

```
    <p class="greeting">Hello, {{ username }}!</p>
```

```
  </div>
```

```
</div>
```

```
// app.component.css
```

```
.container {  
  text-align: center;  
}
```

```
.title {  
  color: green;  
}
```

```
.content {  
  margin-top: 20px;  
}
```

```
.input-field {  
  margin-top: 20px;  
  padding: 10px;  
  border: 1px solid #ccc;  
  border-radius: 5px;  
}
```

```
.greeting {  
  margin-top: 20px;  
}
```

BUILT-IN DIRECTIVES IN ANGULAR

```
// app.component.ts

import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  isHighlighted: boolean = true;
  isItalic: boolean = false;
  textColor: string = 'blue';
  fontSize: number = 18;
  username: string = '';
}
```

```
// app.module.ts

import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    FormsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

BUILT-IN DIRECTIVES IN ANGULAR

Structural Directives

- Structural directives are responsible for manipulating the DOM layout by adding, removing, or manipulating elements based on conditions.
- They are denoted by an asterisk (*) preceding the directive name and are commonly used to alter the structure of the DOM based on conditions.

Built-in Attribute Directives

- **ngIf:** The ngIf directive conditionally includes or removes an element based on a provided expression.

```
<element *ngIf="condition">  
  <!-- Content to display when condition is true -->  
</element>
```

BUILT-IN DIRECTIVES IN ANGULAR

- ❑ **ngFor:** The ngFor directive iterates over a collection and instantiates a template once for each item in the collection.

```
<element *ngFor="let item of items">  
  <!-- Content to repeat for each item -->  
</element>
```

- ❑ **ngSwitch:** The ngSwitch directive is similar to a switch statement in programming languages. It displays one element from a set of elements based on a provided expression.

```
<element [ngSwitch]="expression">  
  <element *ngSwitchCase="value1"> <!-- Content for case 1 --> </element>  
  <element *ngSwitchCase="value2"> <!-- Content for case 2 --> </element>  
  <!-- More ngSwitchCase elements for other cases -->  
  <element *ngSwitchDefault> <!-- Default content --> </element>  
</element>
```

BUILT-IN DIRECTIVES IN ANGULAR

```
// app.component.html

<div class="container">
  <div class="content">
    <div *ngIf="isLoggedIn" class="message">
      Welcome to {{ username }}!
    </div>

    <ul *ngIf="items.length > 0" class="list">
      <li *ngFor="let item of items"
class="list-item">
        {{ item }}
      </li>
    </ul>
```

```
    <div [ngSwitch]="color" class="color-
switch">
      <p *ngSwitchCase="'red'" class="color-
message">Red color selected</p>
      <p *ngSwitchCase="'blue'"
class="color-message">Blue color selected</p>
      <p *ngSwitchCase="'green'"
class="color-message">Green color selected</p>
      <p *ngSwitchDefault class="color-
message">Please select a color</p>
    </div>
  </div>
</div>
```

BUILT-IN DIRECTIVES IN ANGULAR

```
// app.component.css

.container {
  text-align: center;
}
.title {
  color: green;
}
.content {
  margin-top: 20px;
}
.message {
  font-size: 20px;
  color: green;
  margin-bottom: 20px;
}
.list {
  list-style-type: none;
  padding: 0;
}
.list-item {
  font-size: 16px;
  margin-bottom: 5px;
}
.color-switch {
  margin-top: 20px;
}
.color-message {
  font-size: 18px;
  margin-top: 10px;
}
```

```
// app.component.ts

import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  isLoggedIn: boolean = true;
  username: string = 'John';
  items: string[] = ['Item 1', 'Item 2', 'Item 3'];
  color: string = 'red';
}
```

CREATING AN INJECTABLE SERVICE IN ANGULAR

- An injectable service in Angular is a TypeScript class decorated with the `@Injectable()` decorator.
- This decorator provides metadata that allows Angular's dependency injection system to create and manage instances of the service class.
- Syntax:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // or 'any' or specific module
})
export class MyService {
  constructor() { }

  // Service logic here
}
```

CREATING AN INJECTABLE SERVICE IN ANGULAR

Steps to Create An Injectable Service:

- **Step 1:** Create a new Angular project
`ng new injectible-service-demo`
- **Step 2:** Once the project is set up, navigate to the **src/app** directory. Inside it, create a new folder called "**core**". Within the "**core**" folder, create another folder named "**services**".
- **Step 3:** Generate a new service using the Angular CLI after going into services folder using the below command
`ng generate service my-service`

CREATING AN INJECTABLE SERVICE IN ANGULAR

```
// app.component.html
```

```
<div>
  <button (click)="addItem()">Add Item</button>
  <ul>
    <li *ngFor="let item of items">{{ item |
json }}</li>
  </ul>
</div>
```

```
// my-service.service.ts
```

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})

export class MyService {
  private data: any[] = [];

  getData() {
    return this.data;
  }

  addData(item: any) {
    this.data.push(item);
  }
}
```

CREATING AN INJECTABLE SERVICE IN ANGULAR

```
// app.component.ts

import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { MyService } from '../core/services/my-service.service';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, CommonModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
```

```
export class AppComponent {
  title = 'injectible-service-demo';

  items: any[] = [];

  constructor(private myService: MyService) {
    this.items = this.myService.getData();
  }

  addItem() {
    const newItem = { id: Date.now(), value:
'New Item' };
    this.myService.addData(newItem);
    this.items = this.myService.getData();
  }
}
```

ANGULAR MODULES

- Module in Angular refers to a place where you can group the components, directives, pipes, and services that are related to the application.
- The simplest way to create a module is using Angular CLI.
- NgModules are Angular libraries such as FormsModule, HttpClientModule, and RouterModule.
- NgModules can also be used to access third-party libraries such as AngularFire2, Ionic, and Material Design.

ANGULAR MODULES

- There are two methods for creating a module:
 - Using the Angular CLI
 - Manually creating a module

Creating a new module using Angular CLI:

- In most cases this approach is highly recommended, as it is easy, quick and efficient.
- Angular provides a command to generate new Modules.

ANGULAR MODULES

- **Step 1:** Open your terminal or Command prompt and go to the root directory of your Project.
- **Step 2:** Run the following command. In place of '[name]' write your desired module name.
`ng generate module [name]`
- You can also make this command short by using:
`ng g m [name]`

ANGULAR MODULES

Manually creating a new module

- **Step 1:** Go to the app directory, right click on it, select new folder and give it a name.
- **Step 2:** In the new folder right click and select new file and name it "job.module.ts".
- **Step 3:** Now we have to Import ng module from '@angular/core' and add some lines of code.

```
// job.module.ts
import { NgModule } from '@angular/core';
@NgModule({
  declarations: [JobComponent],
  imports: [],
  providers: [],
  bootstrap: [],
})
export class JobModule {
}
```

ANGULAR MODULES

- **Step 4:** Now that we have declared the module, we can import `JobModule` in other modules and use its components there.

Suppose you want to use the **JobModule** in the **AppModule**.

- First, open the **JobModule** file. In the `@NgModule` decorator, make sure to **export the JobComponent** so it can be used in other modules. Your code should look like this:

```
@NgModule({  
  declarations: [JobComponent],  
  imports: [],  
  providers: [],  
  bootstrap: [],  
  exports: [JobComponent],  
})  
export class JobModule {  
}
```

- Once the component is exported, you can **import the JobModule into the AppModule**.
- After that, Angular will recognize `JobComponent`, and you'll see it appear in the code suggestions or be able to use it directly in templates.

DIFFERENCE BETWEEN COMPONENT & MODULE IN ANGULAR

Component	Module
A component is a class with an associated template that defines a view.	A module is a container for a group of related components and directives.
A component is responsible for rendering a view and handling user interactions with the view.	A module can contain services, pipes, and other code that is used by the components that are a part of the module.
A component is defined using a class and a template.	A module is defined using the Angular NgModule decorator.
A component controls a part of the screen called a view.	A module provides a way to group related functionality and keep the code organized.
A component can interact with other components or services.	A module can contain multiple components and directives.

PIPES IN ANGULAR

- The **pipe()** function in Angular is used to chain multiple operators together to transform data.
- It receives an input value, works on it, and gives back a modified result.
- You can recognize pipes in your template expressions by the pipe symbol (|).

`value | pipe1 | pipe2 | ... | pipeN`

- `value`: The input value to be transformed.
- `pipe1, pipe2, ..., pipeN`: Operators or transforms to be applied to the input value.

PIPES IN ANGULAR

- The pipe() function, in Angular provides a variety of functionalities that streamline and enhance data manipulation.
 - **Data Formatting** : Pipes have applications, for organizing data like adjusting dates, numbers, currency and more.
 - **Data Filtering**: Pipes are capable of sorting, searching, or presenting subsets of data by filtering data according to predefined criteria.
 - **Data Transformation**: Data can be transformed by pipes using operations like title case, lowercase, and uppercase, among others.
 - **Localization**: Pipes are handy for managing localization and internationalization of data like translating text or presenting dates, in formats.

PIPES IN ANGULAR

Types of Pipes in Angular

- **Built-in Pipes**

- Angular comes equipped with built in pipes that handle a variety of common formatting duties.

```
<p>Today's date: {{ today | date:'mediumDate' }}</p>
```

- Lets take a look, at some of the ones that are commonly used:

- **DatePipe**-The DatePipe is utilized for date formatting. It enables us to present dates, in styles like short, medium and full.

```
{{ myDate | date: "short" }}
```

- **UpperCasePipe** - This tool changes a text to capital letters. It requires a text, as input and gives back the text in all capital letters.

```
{{ myString | uppercase }}
```

- **LowerCasePipe** - This particular pipe is utilized for changing a string to lowercase. Its functionality resembles that of the UpperCasePipe, except it changes the string to lowercase instead.

```
{{ myString | lowercase }}
```

- **CurrencyPipe** - This tool helps to convert numbers into currency values. You input a number and it gives back a string showing the number, in the desired currency format.

```
{{ myNumber | currency: "USD" }}
```

PIPES IN ANGULAR

- **Custom Pipes**

- When the existing pre-installed pipelines are not enough you can create your customized pipelines. Here is how you can set it up:
 - Create a TypeScript class: Decorate the class with **@Pipe**.
 - To use the **PipeTransform** interface you need to create a **transform()** method that takes the input **value** and any additional **arguments** you specify. In this **transform()** method you will define the logic, for altering the data.

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'Example' })
export class ExamplePipe implements PipeTransform {
  transform(value: string, limit: number = 10): string {
    if (value.length > limit) {
      return value.substring(0, limit) + '...';
    }
    return value;
  }
}
```

<p>{{ longText | example:20 }}</p>

- In Angular templates, you employ pipes using the pipe operator (|) within expressions:
{{ inputValue | pipeName:argument1:argument2:... }}

PIPES IN ANGULAR

- **How to use pipes in your Angular application:**
- **Step 1:** Generate a Component using the following command:
ng generate component display-date
- **Step 2:** Modify the following files:

```
// display-date.component.html
```

```
<p>Today's date (Default format): {{ today }}</p>
<p>Today's date (Short format): {{ today | date:'short' }}</p>
<p>Today's date (Custom format): {{ today |
date:'mm/dd/yyyy' }}</p>
```

```
// display-date.component.ts
```

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-display-date',
  templateUrl: './display-date.component.html',
  styleUrls: ['./display-date.component.css']
})
export class DisplayDateComponent implements OnInit {
  today = Date.now();
}
```